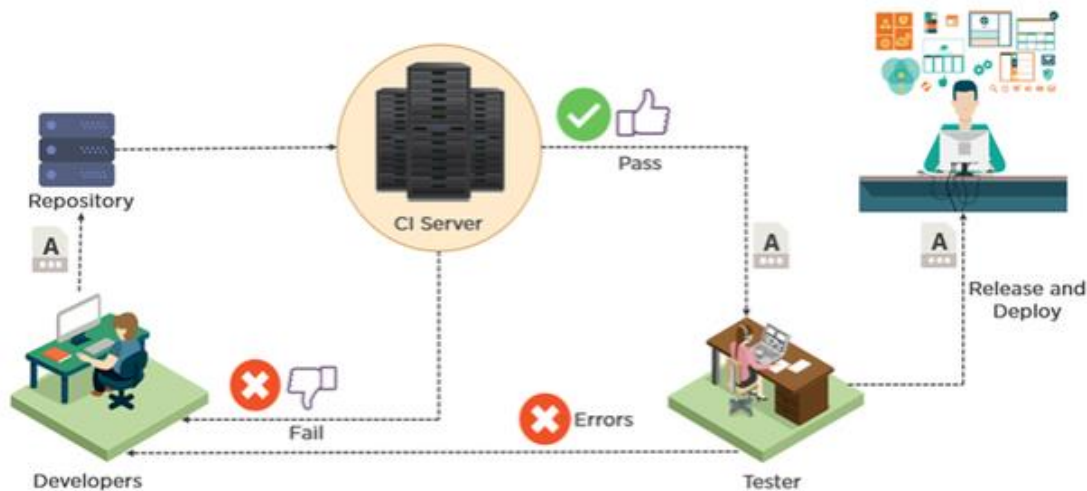


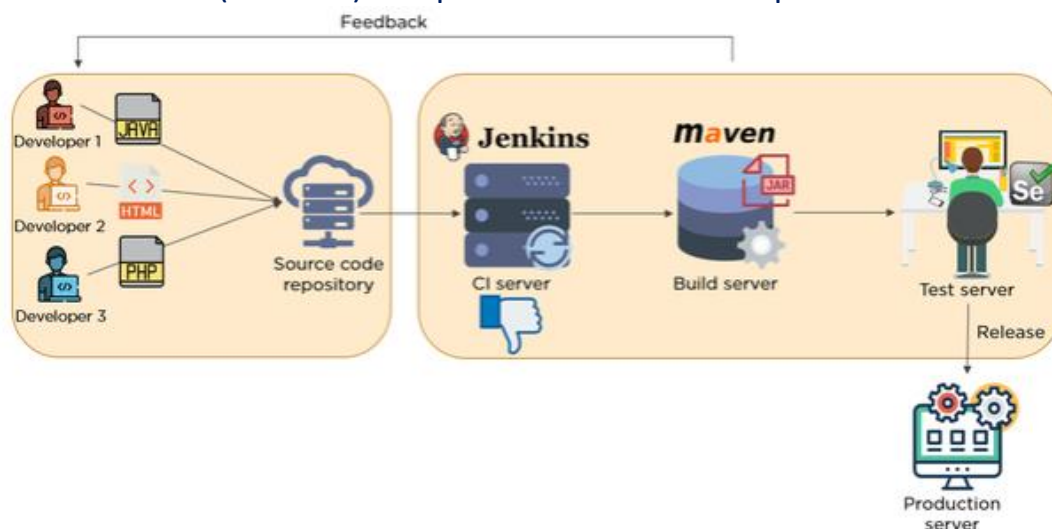
Jenkins

Jenkins is an open source continuous Integration server written in java that allows continuous development , build,test and deployments of codes.



Once application is ready our target is to deploy it in different environments, so that it will be available for n number of users.

We don't deploy the source code, first we convert the source code into executable format (artifacts) this process is called build process.



We don't deploy directly in to the production before moving to production there are different environment where we test our application, just like covid19 vaccine testing phases.

Build Code: Is the process of converting source code to executable/artifacts.

java --.jar/.war/. ear

net -- .exe/.dll/msi

python – pybuilder

How to build code? Using the build tool

java -- **maven**/ant/Gradle

net -- vs/msbuild

Build code process: involves various stages

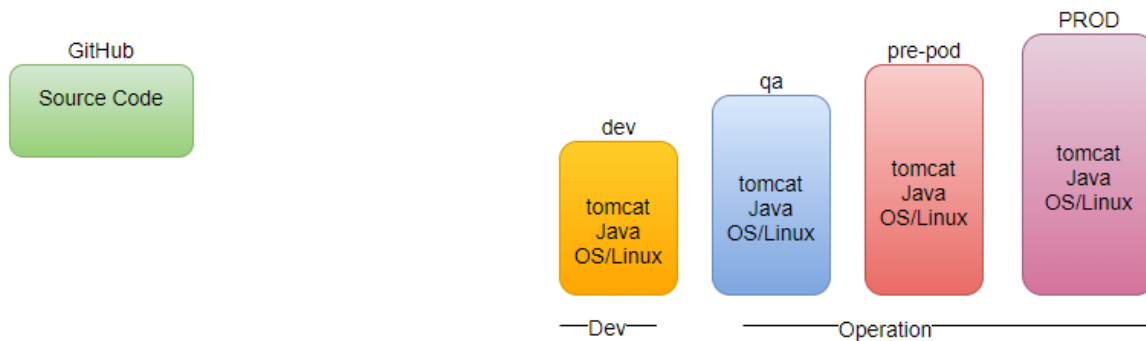
We can directly package sc to artifacts, but it is recommended to go through various stages

code review: static code analysis

unit test: developer level testing

code coverage: how much testing covered.

package: convert sc to executable



Maven:

What is Maven?

Maven is a **build tool** that is used to compile, validate codes and analysis the test-cases in the code.

Maven is also known as dependency management tool.

In maven there are three repositories.

1. local repository
2. center repository
3. remote repository

In first time it stores the file in **.m2** folder

How maven understand our code?

pom.xml (project object model) # developer writes the pom.xml

GAV

groupid: uniquely Identify your project.

artifactid: name of the package (.jar) without the version

versionid: version of the .jar

packaging tag

default: .jar

maven has three defined Lifecycles:

- default: build
- clean
- site:

Goal:

- Validate
- Compile
- Test
- Package
- Install



Maven creates a **folder target** # output of maven command

Each Lifecycle has their own phases

default: build

- validate
- compile
- test
- package
- integration-test
- verify
- install
- deploy

clean

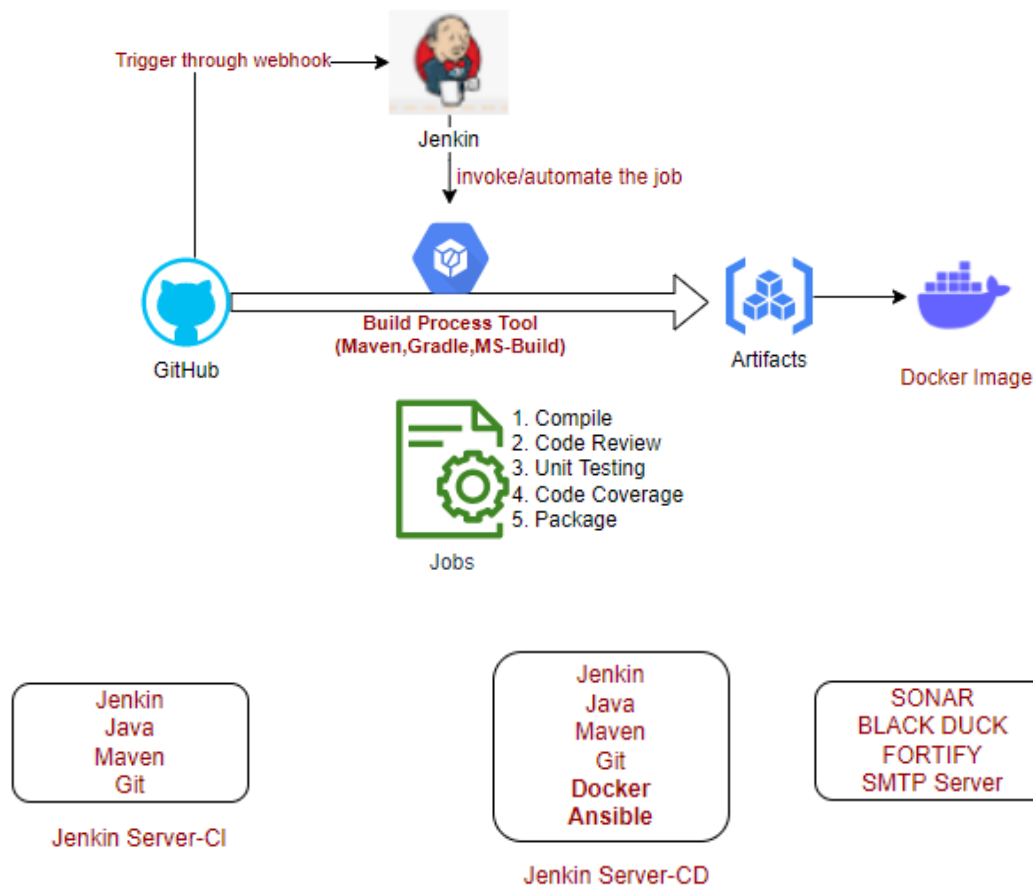
- pre-clean
- clean
- post-clean

Maven is built and dependency management tool for Software projects. It provides a DSL(.pom) to define, build, package and publish the project artifacts To put it in a simple word, it's a tool to define how a project is built and how its dependencies are managed in order to build the project artifacts i.e., JAR, WAR or EAR of a Java project.

All the above steps performed by maven, then what is Jenkins? why we need Jenkins?

Jenkins: Build automation server, helps automating the build code process.

- We integrate Git with Jenkins
- We integrate java with Jenkins
- We integrate maven with Jenkins

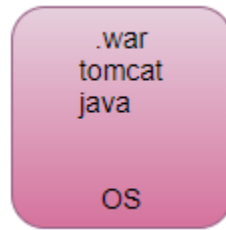


Job: Activities which we create.

Build: Execution of created jobs

We perform various stages of build code process continuously, that's why we call Jenkins as continuous integration server.

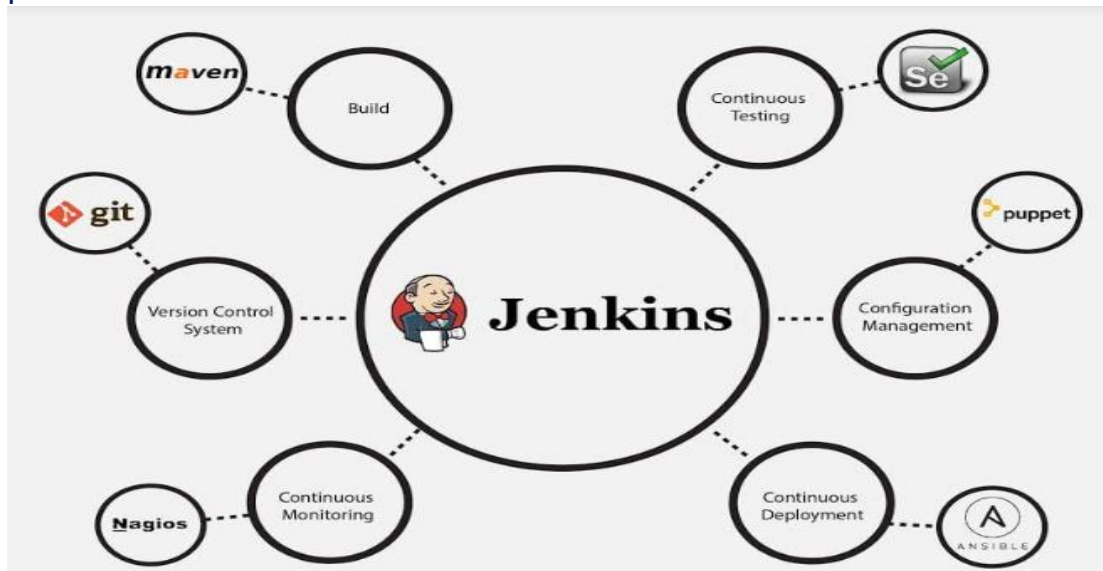
If we execute the various stages of build code process in a sequence manner that's called continuous integration pipeline (CI PIPELINE)



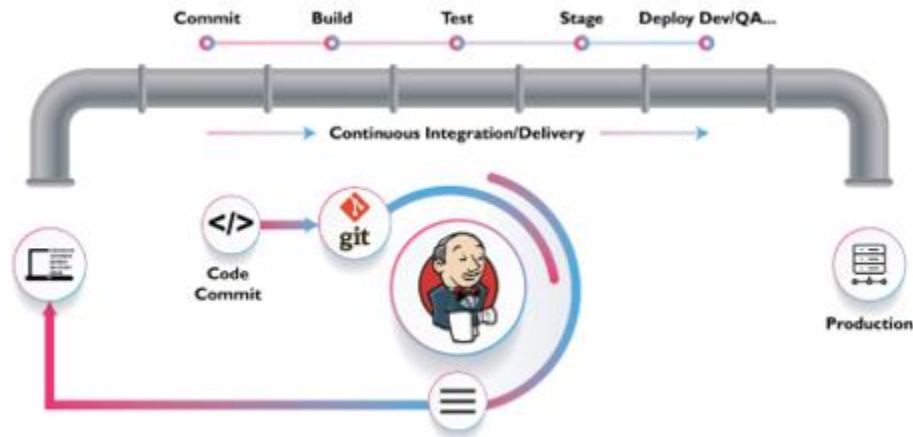
When we feel its **repetitive work go for automation**, where Jenkins comes in picture called **build automation server**, also called **continuous Integration server**. Jenkins is also called dummy tool it can't do anything its own, we have to give instruct to Jenkins hey go to git server take a latest code, hey Jenkins go and talk to build server make a build of latest code. This also called continuous Integration pipeline, it is a mediator collaborate with other tools also called orchestration tool this pipeline always executed whenever there is a commit in GitHub.

IaaS- Ansible, terraform, CloudFormation.

Jenkins's server repeats the tasks for you. It helps us to automate the entire build process.



Bigger Picture Role of Jenkins in DevOps?



Installation:

Install java

Install Jenkins

Install Maven

JAVA_HOME: /usr/lib/jvm/java-8-openjdk-amd64

MAVEN_HOME: /opt/apache-maven-3.6.3

JENKINS_HOME: /var/lib/Jenkins

Jenkins is a tool integrate all tools together

Home->Manage Jenkins-> Global tool configuration

- **Java_Home:** /usr/lib/jvm/java-8-openjdk-amd64
- **Git:** which git
- **Maven_Home:** /opt/apache-maven-3.6.3

When we install a Jenkins, Jenkins run as a service/process.

service jenkins status

Why and how to open firewall ports in GCP?

Open the browser and run the Jenkins at port 8080, Open the browser and enter <Public Ip>:8080



This site can't provide a secure connection

34.93.137.14 sent an invalid response.

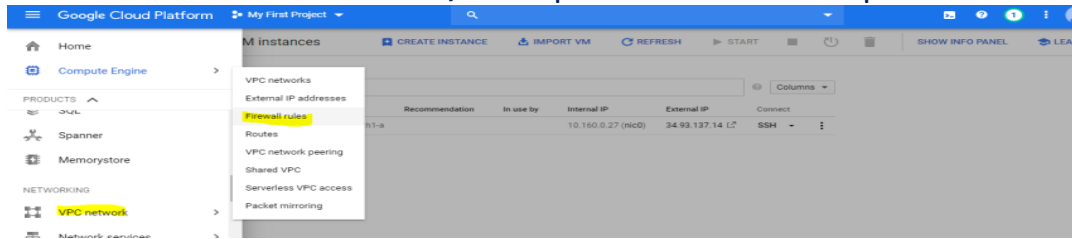
[Try running Windows Network Diagnostics.](#)

ERR_SSL_PROTOCOL_ERROR

Reload

How?

VPC network-> firewall rules, and open firewall rule for port 8080



<Public Ip>:8080

Getting Started

Unlock Jenkins

To ensure Jenkins is securely set up by the administrator, a password has been written to the log ([not sure where to find it?](#)) and this file on the server:

`/var/lib/jenkins/secrets/initialAdminPassword`

Please copy the password from either location and paste it below.

Administrator password

`cat /var/lib/jenkins/secrets/initialAdminPassword`

Note: Replace the public Ip with private Ip

Jenkins is just a dummy tool; you must instruct to jenkins what to do.

How?

create a job (Item).

Without Jenkins, All the code is in our local, and we have to do the following steps manually.

- Compile
- Code Review
- Unit Testing
- Code Coverage
- Package

```
root@instance-1:~/samplejavaapp#  
root@instance-1:~/samplejavaapp# /opt/apache-maven-3.6.3/bin/mvn package  
[INFO] Scanning for projects...
```

How we automate through Jenkins?

Jenkins Console

General: Description (Good Description What your job is going to do)

General Source Code Management Build Triggers Build Environment Build Post-build Actions

[Plain text] [Preview](#)

- ☐ Discard old builds
- ☐ GitHub project
- ☐ This build requires lockable resources
- ☐ This project is parameterized
- ☐ Throttle builds
- ☐ Disable this project
- ☐ Execute concurrent builds if necessary
- ☐ Quiet period
- ☐ Retry Count
- ☐ Block build when upstream project is building
- ☐ Block build when downstream project is building
- ☐ Use custom workspace

Display Name

- ☐ Keep the build logs of dependencies

Source Code Management: job depends on some Repositories (Git Hub)

Source Code Management

- ☒ None
- ☐ Git
- ☐ Subversion

Build Triggers: You define when job will be triggered (certain time, event)

Remotely- token

<JENKINS_URL>/job/1-compile/build?Token= <TOKEN_NAME>

Build Triggers

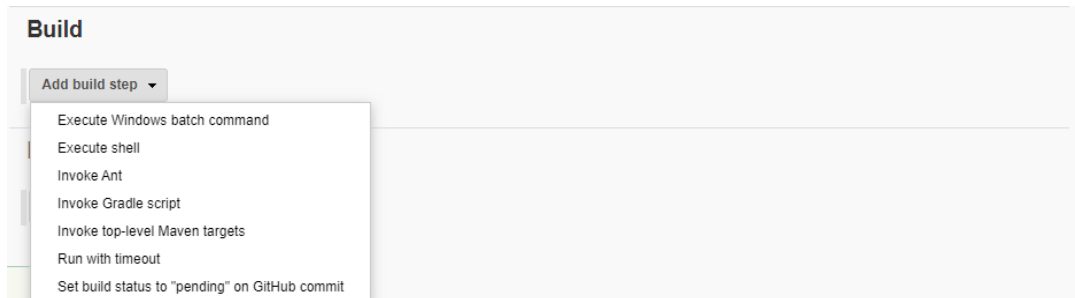
- ☐ Trigger builds remotely (e.g., from scripts)
- ☐ Build after other projects are built
- ☐ Build periodically
- ☐ GitHub hook trigger for GITScm polling
- ☐ Poll SCM

Build Environment: Where you define environment Variable

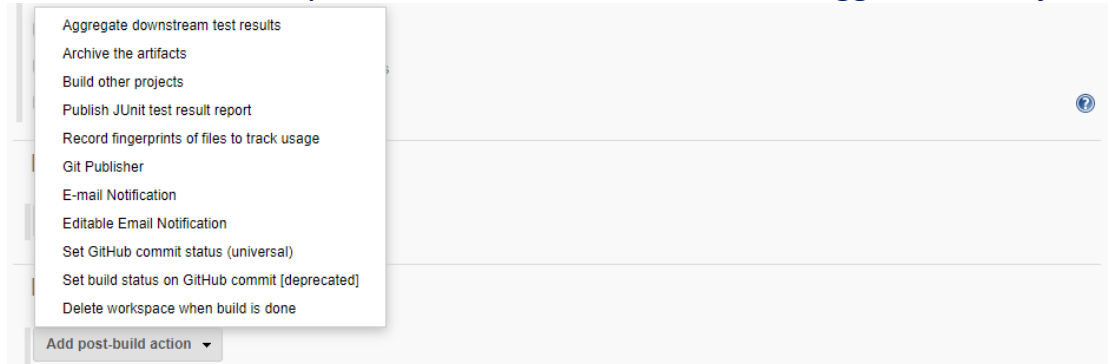
Build Environment

- ☐ Delete workspace before build starts
- ☐ Use secret text(s) or file(s)
- ☐ Abort the build if it's stuck
- ☐ Add timestamps to the Console Output
- ☐ Inspect build log for published Gradle build scans
- ☐ With Ant

Build: What has to be done by this job



Post-build action: Once your build done, what after that trigger another job?



How to create a new job in Jenkins?

Project Type: Freestyle/Pipeline/...

General:

Source Code Management:

Build Trigger:

Build Environment:

Build:

Post Build Actions:

Example: Test Job

Home Page



Project Type: Free Style

General: Very Simple Job

Source Code Management:

Build Trigger:

Build Environment:

Build: Execute Shell (echo "hello world")

Post Build Actions:

Home Page->Test Job-> Build Now

Jenkins never override previous build, always creates new build.

Default Job created in /var/lib/Jenkins/workspace/<Name of Job>

System Configuration:

Manage Jenkins->Global Tool Configuration:

The screenshot shows the Jenkins 'Global Tool Configuration' page for 'JDK'. It features a sidebar with 'JDK installations' and a main form area. The form includes an 'Add JDK' button, a 'Name' field (containing 'java1.11'), a 'JAVA_HOME' field (containing '/usr/lib/jvm/java-11-openjdk-amd64'), and an 'Install automatically' checkbox (checked). Below the form is a table listing existing JDK installations. On the right side, there are 'Delete Installer' and 'Delete JDK' buttons.

Name	Version	Install automatically	Actions
java1.11		☒	Delete Installer Delete JDK

Git

List of JDK installations on this system

Git installations

Git

Name: Default

Path to Git executable: git

☐ Install automatically

Delete Git

Add Git

Maven

Maven installations

Add Maven

Maven

Name: maven3.6

MAVEN_HOME: /opt/apache-maven-3.6.3

☒ Install automatically

Delete Maven

Add Maven

List of Maven installations on this system

What is plugin?

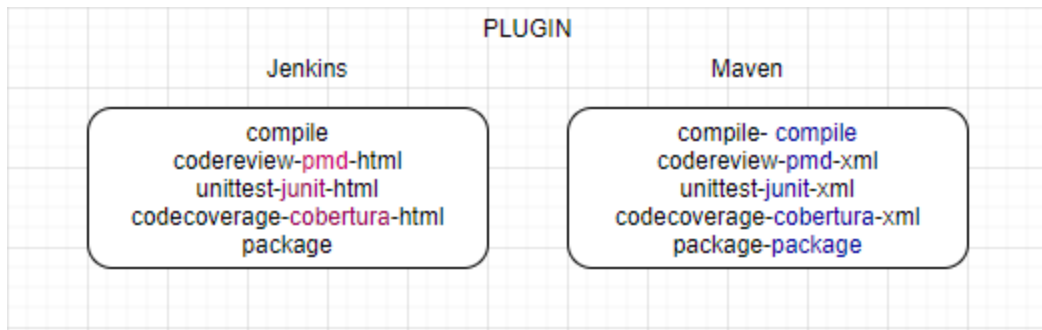
Enhance the functionality of basic S/w.

mobile phone comes with basic functionalities, to add more features we have to go to play store and add additional features, similarly jenkins comes with basic functionalities, just like maven comes with basic feature with compile, now we want code review but **code review functionality not a default feature of maven**, it needs some plugin to extend the functionality of the tools, need **pmd (program mistake detector) plug-in** to enhance the functionality of maven to do the code review, will take from the pom.xml.

Similarly, **code coverage** is done by another plugin cobertura.

Similar Jenkins comes with various functionalities and can enhance its functionalities by adding corresponding plug-in. various integration can be done with Jenkins to perform various tasks.

Jenkins is very popular for its plugin itself; plugin helps to Jenkins to enhance it up to any level.



UI shown, based on added plugins, after adding Plugin some more block will be visible e.g., after adding audit trial Plug In, Logger block will be visible.

Jenkins enhanced its functionality with the help of plug-in, Jenkin support approx. (1500+) plug-in. (**plugins.jenkins.io**)

Manage Jenkins-> Manage Plugins

Global Variables

/env-vars.html

Use Case: Understand how manually application working?

Compile->Test->Review->Package->Deploy

steps to clone and compiling samplejavaapp manually

mkdir tmpapplication

cd tmpapplication

git clone <https://github.com/NeerajVohra/samplejavaapp.git>

mvn compile

mvn test

Continuous Integration (Using Jenkins)

Compile->Test->Review->Package (CI)

step1: Create a new job ApplicationCompile

Project Type: Free Style

General: Pull the Source Code from GitHub and Compile

Source Code Management: Git URL

<https://github.com/NeerajVohra/samplejavaapp.git>

Git understands, it has to clone the remote repository.

/var/lib/jenkins/workspace/<job name> (keeps output of job in /target folder)

Build Trigger:

Build Environment:

Build: **invoke top-level maven targets** (option will available once we install maven in Jenkins (global tool configuration). Uninstall maven from system (apt-get remove --purge maven)

Goal-> compile

Build

Invoke top-level Maven targets

Maven Version

Goals

Advanced...

Post Build Actions: NA

/var/lib/jenkins/workspace/<job>/target (contains output generated by job)

Workspace can be viewed from console.

Back to Dashboard

Status

Changes

Workspace

Wipe Out Current Workspace

Build Now

Configure

Delete Project

Rename

Build History trend ^

Workspace of job2-codereview on master

.git

build

gradle/wrapper

src

target

.gitignore

build.gradle

Dockerfile

gradlew

gradlew.bat

Jenkinsfile

jenkinsfile-k8s

jenkinsfile-win

pom.xml

README.md

settings.gradle

Nov 13, 2020 3:20:08 AM

Nov 13, 2020 3:20:08 AM

Nov 13, 2020 3:20:08 AM

Nov 13, 2020 3:20:08 AM

Nov 13, 2020 3:20:08 AM

Nov 13, 2020 3:20:08 AM

Nov 13, 2020 3:20:08 AM

Nov 13, 2020 3:20:08 AM

Nov 13, 2020 3:20:08 AM

Nov 13, 2020 3:20:08 AM

Nov 13, 2020 3:20:08 AM

Nov 13, 2020 3:20:08 AM

9 B

1.71 KB

117 B

5.63 KB

2.87 KB

1.36 KB

821 B

1.59 KB

19.96 KB

31 B

93 B

view

view

view

view

view

view

view

view

view

view

view

Full file view

Back to Project

Status

Changes

Console Output

View as plain text

Edit Build Information

Delete build '#1'

Git Build Data

Build #1 (Jan 9, 2021 5:38:09 PM)

No changes.

Started by user [admin](#)

git

Revision: 2a035fcb6cf770dcacf9f9be3555ad4ed172b38e

refs/remotes/origin/master

Back to Project

Status

Changes

Console Output

View as plain text

Edit Build Information

Delete build #1

Git Build Data

Console Output

```

Started by user admin
Running as SYSTEM
Building in workspace /var/lib/jenkins/workspace/package
The recommended git tool is: NONE
No credentials specified
Cloning the remote Git repository
Cloning repository https://github.com/lerndevops/samplejavaapp
> git init /var/lib/jenkins/workspace/package # timeout=10
Fetching upstream changes from https://github.com/lerndevops/samplejavaapp
> git --version # timeout=10
> git --version # "git version 2.30.0"
> git fetch --tags --force --progress -- https://github.com/lerndevops/samplejavaapp :refs/heads/*:refs/remotes/origin/* # timeout=10
> git config remote.origin.url https://github.com/lerndevops/samplejavaapp # timeout=10
> git config --add remote.origin.fetch +refs/heads/*:refs/remotes/origin/* # timeout=10
Avoid second fetch
> git rev-parse refs/remotes/origin/master^{commit} # timeout=10
Checking out Revision: 2a035f0b8c9f70a0ac6f070e355a04ed172b38e (refs/remotes/origin/master)
> git config core.sparsecheckout # timeout=10
> git checkout -f 2a035f0b8c9f70a0ac6f070e355a04ed172b38e # timeout=10
Commit message: "Update README.md"
First time build. Skipping changelog.
[package] $ /opt/apache-maven-3.6.3/bin/mvn package
[INFO] Scanning for projects...
[WARNING]

```

step2: Create a new job ApplicationReview.

Project Type: Free Style

General: Pull the Source Code from GitHub and Review pmd (program mistake detector)

Source Code Management: Git URL

<https://github.com/NeerajVohra/samplejavaapp.git>

Build Trigger: Build after other projects are built

Build Environment:

Build: invoke top-level maven targets

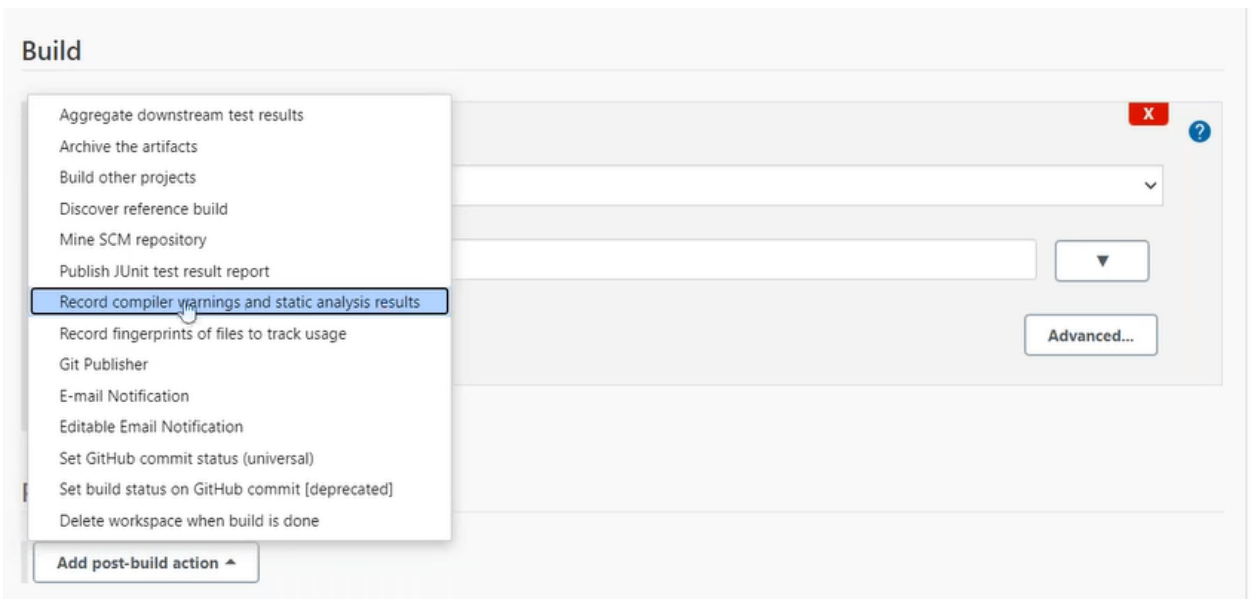
Goal-> pmd:pmd -P metrics pmd:pmd

Maven generates the report in xml format, we can instruct Jenkins, **hey! Jenkins, please convert the generated file in UI format.** (add the Jenkins Plugin for the same).

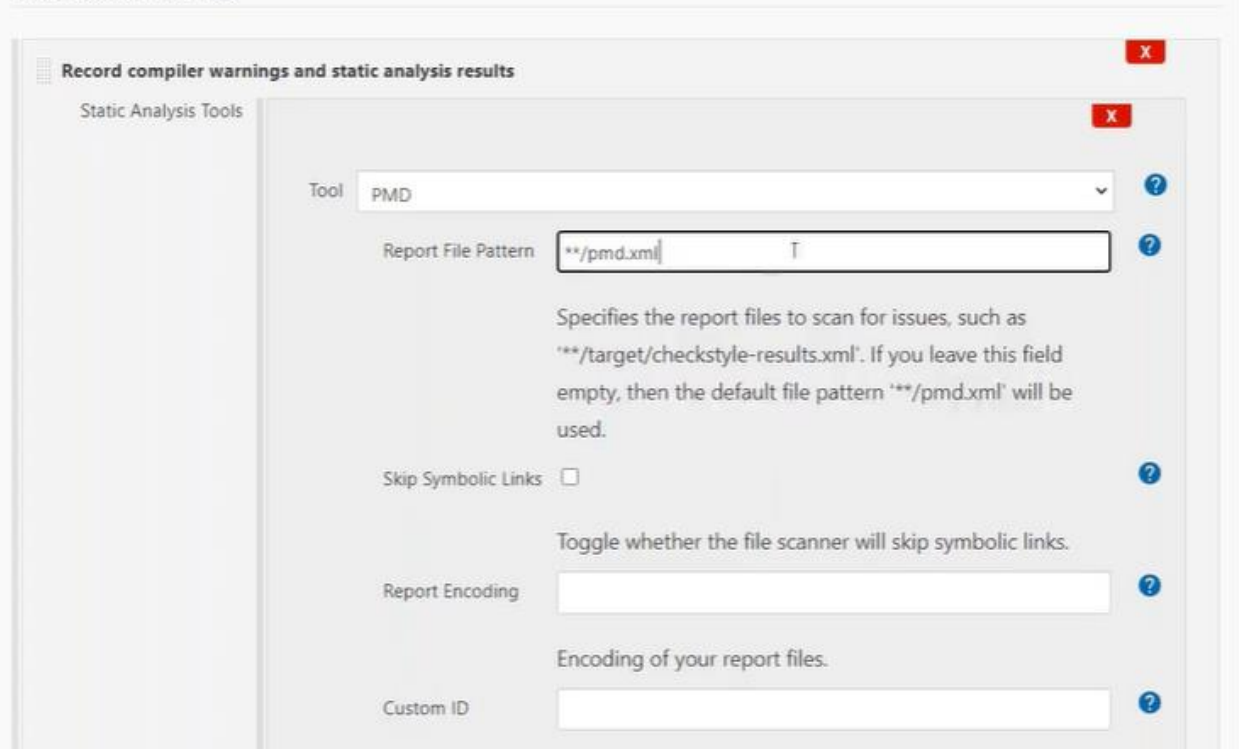
Post Build Actions: Record Compiler Warnings and static analysis Result (If option not available then first install the **Warnings Next Generation Plugin.**

Click on Add post-build action (select Record compiler warnings and static analysis results)

Post Build Action:



Post-build Actions



**** /pmd.xml** // ** means /target folder for Jenkins
/var/lib/jenkins/workspace/<job>/target (contains output generated by job)

step3: Create a new job ApplicationTest.

Project Type: Free Style

General: Pull the Source Code from GitHub and Unit Test

Source Code Management: Git URL

<https://github.com/NeerajVohra/samplejavaapp.git>

Build Trigger:

Build Environment:

Build: invoke top-level maven targets

Goal-> test

Post Build Actions: publish Junit test result report

Test report XMLs: target/surefire-reports/*.xml

/var/lib/jenkins/workspace/<job>/target /surefire reports (contains output generated by job)

step4: Create a new job ApplicationMetricsCheck.

Project Type: Free Style

General: Pull the Source Code from GitHub and code coverage

Source Code Management: Git URL

<https://github.com/NeerajVohra/samplejavaapp.git>

Build Trigger:

Build Environment:

Build: invoke top-level maven targets

Goal-> cobertura: cobertura -Dcobertura.report.format=xml

Post Build Actions: publish Cobertura Coverage Report (If option not available then first install the Cobertura plugin) from home->manage jenkins-> manage plugin

Cobertura xml report pattern target/site/cobertura/coverage.xml

/var/lib/jenkins/workspace/<job>/target (contains output generated by job)

step5: Create a new job ApplicationPackage.

Project Type: Free Style

General: Pull the Source Code from GitHub and create .war

Source Code Management: Git URL

<https://github.com/NeerajVohra/samplejavaapp.git>

Build Trigger:

Build Environment:

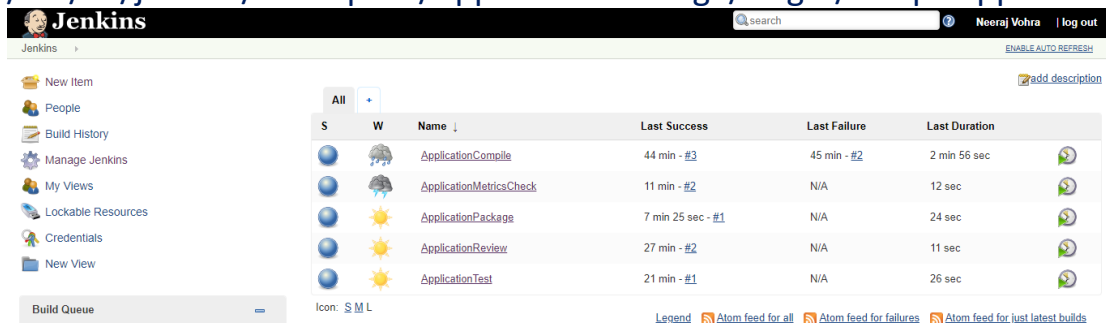
Build: invoke top-level maven targets

Goal-> package

Post Build Actions:

/var/lib/jenkins/workspace/<job>/target (contains output generated by job)

/var/lib/jenkins/workspace/ApplicationPackage/target/sampleapp.war



The screenshot shows the Jenkins dashboard with a sidebar on the left containing links like 'New Item', 'People', 'Build History', 'Manage Jenkins', 'My Views', 'Lockable Resources', 'Credentials', and 'New View'. The main area displays a table of jobs with columns for status (S for Success, W for Warning), name, last success, last failure, and last duration. The jobs listed are ApplicationCompile, ApplicationMetricsCheck, ApplicationPackage, ApplicationReview, and ApplicationTest. ApplicationPackage is highlighted with a yellow background.

S	W	Name ↓	Last Success	Last Failure	Last Duration
●	●	ApplicationCompile	44 min - #3	45 min - #2	2 min 56 sec
●	●	ApplicationMetricsCheck	11 min - #2	N/A	12 sec
●	●	ApplicationPackage	7 min 25 sec - #1	N/A	24 sec
●	●	ApplicationReview	27 min - #2	N/A	11 sec
●	●	ApplicationTest	21 min - #1	N/A	26 sec

All jobs running properly independently.

- ApplicationCompile
- ApplicationReview
- ApplicationTest
- ApplicationMetricsCheck
- ApplicationPackage

How to integrate all Independent Jobs?

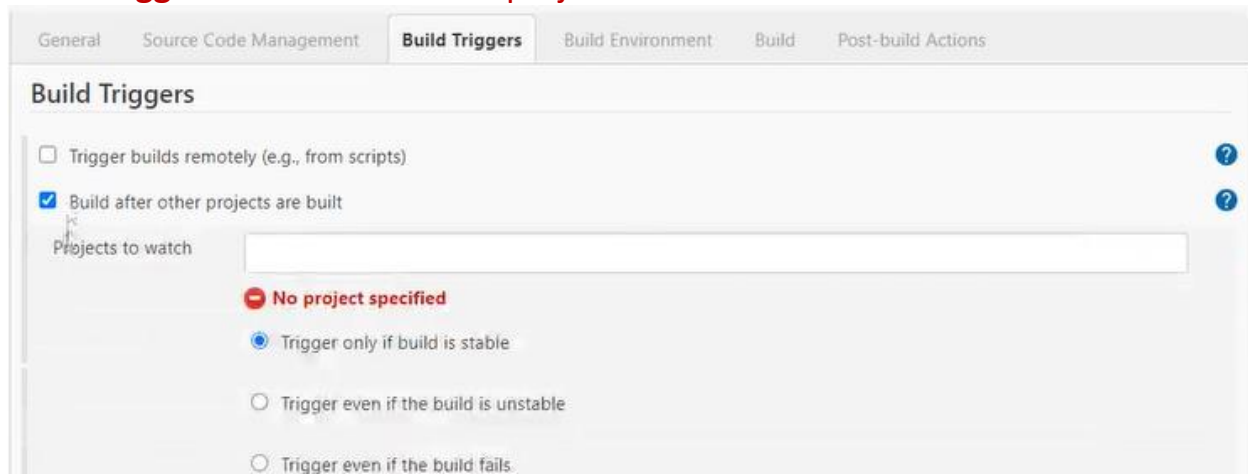
Update the configuration of each project->post build actions of each job

Post Build Actions->build other projects

(or)

Update the configuration of each project-> build trigger actions of each job

Build Triggers->build after other projects are built

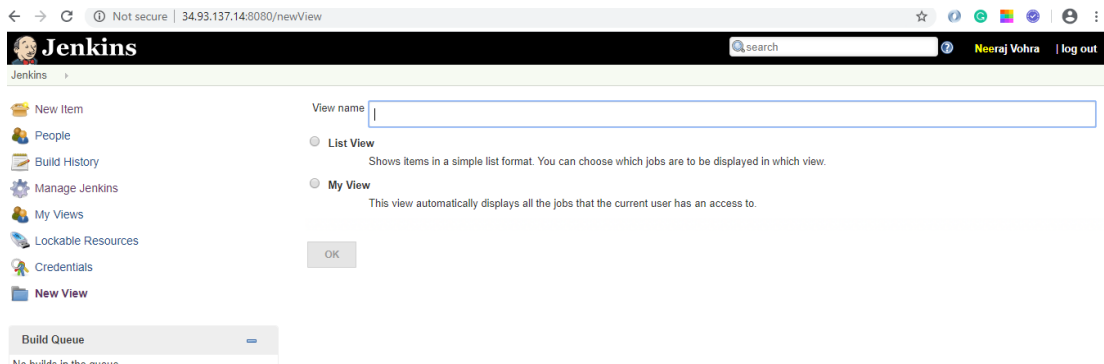


The screenshot shows the 'Build Triggers' configuration page in Jenkins. It has tabs for 'General', 'Source Code Management', 'Build Triggers', 'Build Environment', 'Build', and 'Post-build Actions'. The 'Build Triggers' tab is active. The configuration includes a checkbox for 'Trigger builds remotely (e.g., from scripts)' which is unchecked. The 'Build after other projects are built' checkbox is checked. Below it is a text field for 'Projects to watch' which is empty. A red error message 'No project specified' is displayed. There are three radio buttons for triggering conditions: 'Trigger only if build is stable' (selected), 'Trigger even if the build is unstable', and 'Trigger even if the build fails'.

How wrap all Jobs around pipeline?

Click on + sign

No View for Pipeline



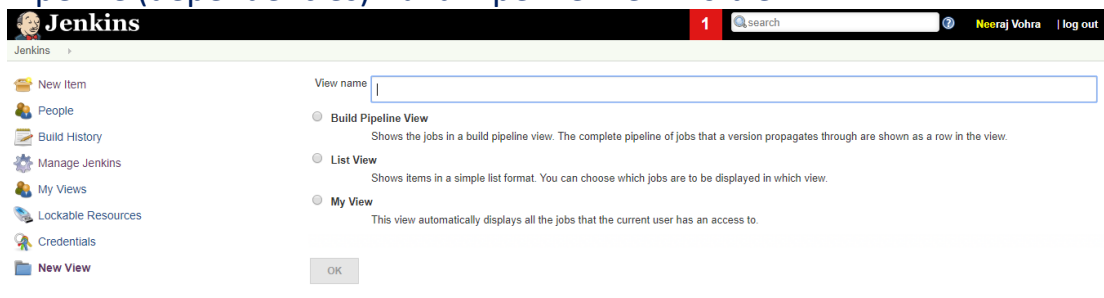
How to visualize dependencies which job depends on which job? Dependencies between jobs are configured as upstream and downstream configure in Jenkins? job3 is dependent on job2, job2 is upstream (previous job) for job3, and Job4 is downstream (next job) for job3.

Pipeline (Automate CI process)

Home->Manage Jenkins-> Manage Plugins (add **build pipeline** plugin)

Click on + sign

Pipeline (dependencies) Build Pipeline View visible

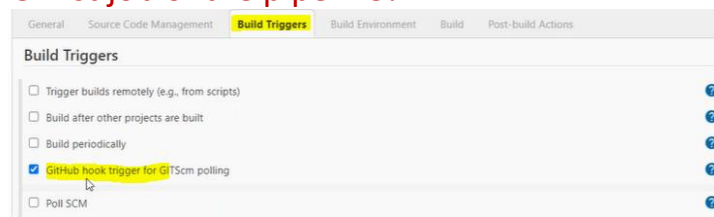


step1: view name: sampleapp_CI

step2: select pipeline tab-> Pipeline Flow (Upstream (previous job)/downstream (next job) config)

Select the initial job (ApplicationCompile)

How to trigger the first job of the pipeline?



Poll SCM

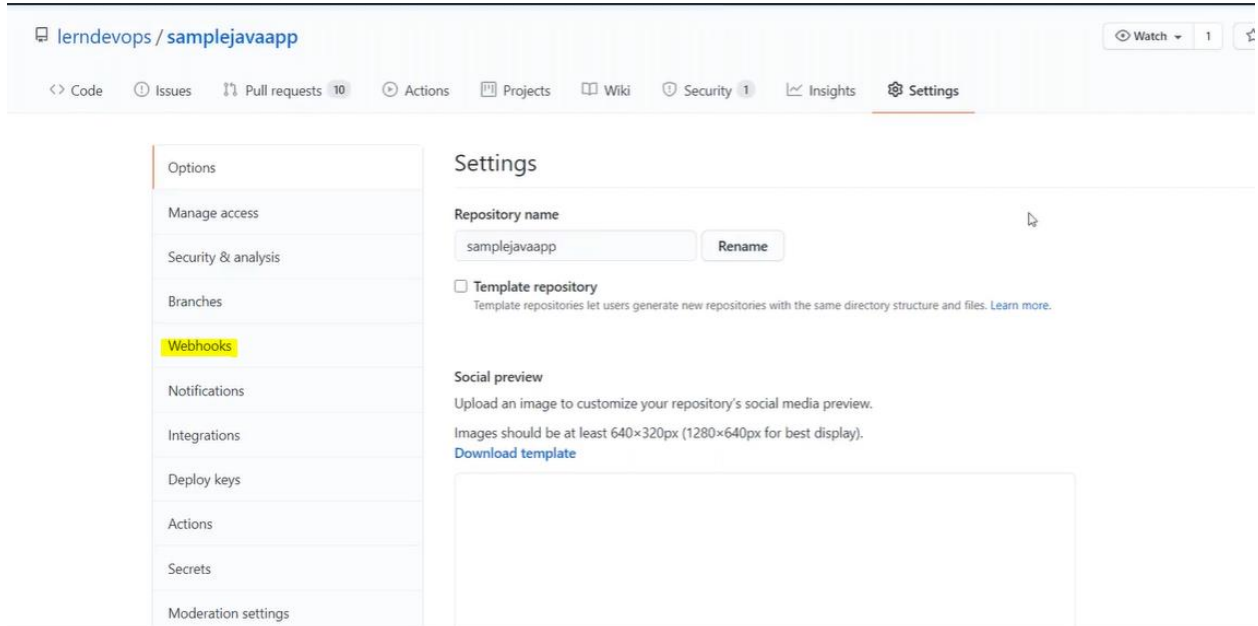
Jenkins's server poll the GitRepo, as per schedule. If there is any change in repository then Jenkins will execute the Job.

Build trigger Webhook?

Git Itself invokes the job, in case of any push.

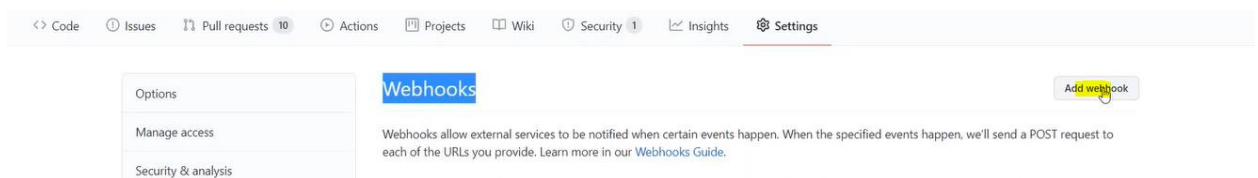
step1: select the option GitHub hook trigger for GitSCM pooling

step2: In GitHub, go to the project as owner of the Project-> setting

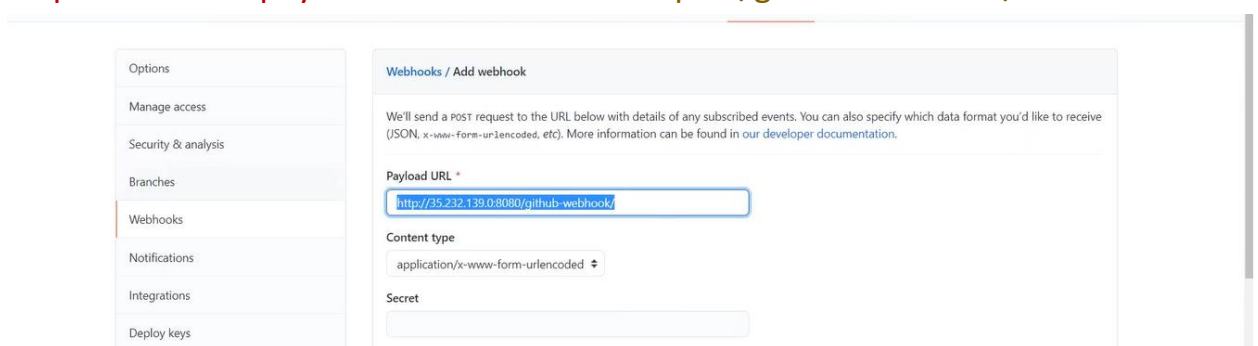


step3: select the webhooks

step4: click on add webhook



step5: enter the payload URL: 'Jenkins URL: port/github-webhook/'



step6: select the event when webhook invoked

The image shows two screenshots from the Jenkins web interface. The top screenshot is the 'Webhook' configuration page. On the left, there's a sidebar with 'Actions', 'Secrets', and 'Moderation settings'. The main area is titled 'Which events would you like to trigger this webhook?'. It has three radio buttons: 'Just the push event.', 'Send me everything.' (which is selected), and 'Let me select individual events.'. Below these, there's a section for 'Active' webhooks. A green 'Add webhook' button is visible. The bottom section, 'Let me select individual events.', contains several checkboxes for specific events: 'Branch or tag creation', 'Branch or tag deletion', 'Check runs', 'Check suites', 'Code scanning alerts', 'Collaborator add, remove, or changed', 'Commit comments', and 'Deploy keys'. The bottom screenshot shows the 'Webhooks' page in the Jenkins UI. It has a sidebar with 'Options', 'Manage access', 'Security & analysis', 'Branches', and 'Webhooks'. The main area is titled 'Webhooks' and contains a list of webhooks. One webhook is shown with the URL 'http://35.232.139.0:8080/github-...' and the event type '(pull_request,pull_request,...)'. There are 'Edit' and 'Delete' buttons for each webhook. The bottom screenshot shows the 'Build Pipeline' view. It has a header with the Jenkins logo, a search bar, and a user profile 'Neeraj Vohra'. Below the header, there's a 'Build Pipeline' section with a toolbar containing 'Run', 'History', 'Configure', 'Add Step', 'Delete', and 'Manage'. The pipeline itself is a sequence of six steps: '#4 ApplicationCompile', '#3 ApplicationReview', '#2 ApplicationTest', '#3 ApplicationMetricsCheck', and '#2 ApplicationPackage'. Each step is represented by a green box with a status icon, a timestamp, and a duration. The steps are connected by arrows, indicating a sequential flow.

Email Configuration

step1: Home-> manage Jenkins-> Configure Systems (servers we don't install in Jenkins running somewhere else just configuring in Jenkins through configure system).

SMTP servers maintained by system team can get configuration information from them.

Extended E-mail Notification

SMTP server	smtp.gmail.com
SMTP Port	465
SMTP Username	leaddevops
SMTP Password	Concealed Change Password
Use SSL	☒
Advanced Email Properties	
Default user e-mail suffix	@gmail.com
Charset	UTF-8
Additional accounts	Add
Default Content Type	Plain Text (text/plain)

5 parameters are mandatory (SMTP server, SMTP port, SMTP username, SMTP password, check the Use SSL)

maximum attached size -1 (unlimited)

default Jenkins's variables

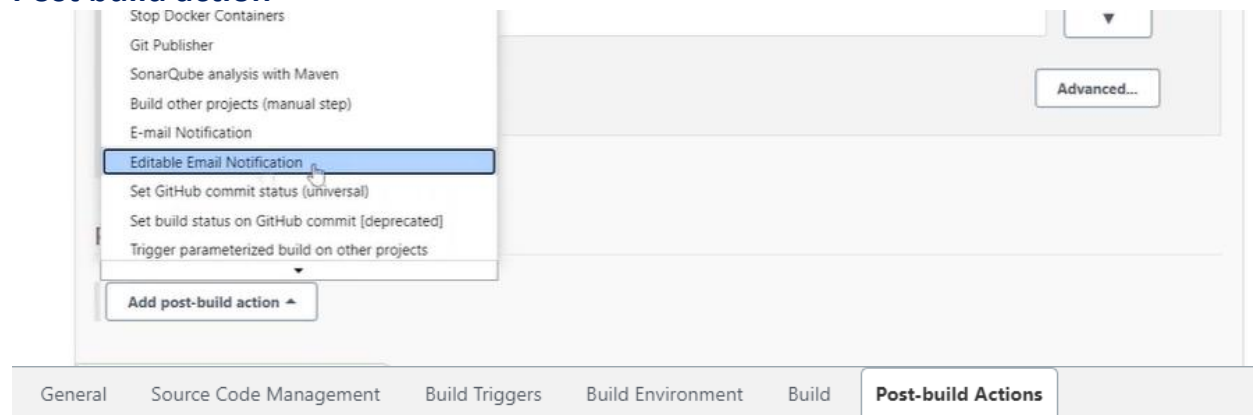
Default Subject	\$PROJECT_NAME - \$BUILD_STATUS!
Maximum Attachment Size	1
Default Content	\$PROJECT_NAME - \$BUILD_STATUS: kljfsdlkj

step3: Click on Default Triggers

Default Post-send Script	
Additional groovy classpath	Add
Enable Debug Mode	☐
Require Administrator for Template Testing	☐
Enable watching for jobs	☐
Allow sending to unregistered users	☐
Content Token Reference	Default Triggers...

step4: After doing the settings at configuration level, go to Job. (you want to send the mail)

Post build action



Post-build Actions

Editable Email Notification X

?
☐ Disable Extended Email Publisher ?
Allows the user to disable the publisher, while maintaining the settings

Project From

Project Recipient List ?

Comma-separated list of email address that should receive notifications for this project.

Project Reply-To List ?

Attachments ?

Can use wildcards like 'module/dist/**/*.zip'. See the [@includes of Ant fileset](#) for the exact format. The base directory is [the workspace](#).

Attach Build Log ?

Attach Build Log
Do Not Attach Build Log
Attach Build Log
Compress and Attach Build Log

Content Token Reference ?

step5: In google account disable the security option

How do I remove Google security settings?

- Turn off 2-Step Verification
- Open your Google Account.
- In the "Security" section, select 2-Step Verification. You might need to sign in.
- Select Turn off.
- A pop-up window will appear to confirm that you want to turn off 2-Step Verification. Select Turn off.

Where is security option in Google Account?

- Click on the blue My Account button to access your account's settings.

Jenkins's security configuration?

- Authentication
- Authorization

Home-> manage Jenkins-> configure Global Security.

Home-> manage Jenkins-> manage user (By default Jenkins own user database)

Authentications:

- Jenkins own user database
- LDAP
- Unix user/group database

Jenkins stores everything in its own database by default.

`/var/lib/jenkins/users`

step1: create user.

manage jenkins->manage users-> create user.

`/var/lib/jenkins/users`

`ls -l`

`more users.xml`

step2: by default, created user have admin privileges.

step3: restrict the access to newly created user.

- manage jenkins-> configure global security (authorization).
- Anyone can do anything.
- Legacy mode.
- Logged-in users can do anything.
- Matrix-based access
- Project-based matrix authorization strategy

Configure Global Security

Authentication

Security Realm

☐

Disable remember me

☒

Delegate to servlet container

☒

Jenkins' own user database

☐

Allow users to sign up

☐

LDAP

☐

Unix user/group database

☐

None

Authorization

Strategy

Authorization

☐ Anyone can do anything

☐ Legacy mode

☒ Logged-in users can do anything

☐ Allow anonymous read access

☐ Matrix-based security

☒ Matrix-based security

User/group	Overall	Credentials	Agent	Job	Run	View	SCM	Lockable Resources			
	Administrator	Create Delete	Update View	Configure Cancel Build	Discover Delete Create Move	Workspace Read Delete	Configure Update Replay	Tag Read Delete Create	Reserve	Unlock	View
Anonymous Users	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐
Authenticated Users	☒	☒	☐	☐	☐	☐	☐	☐	☐	☐	☐
admin	☒	☒	☐	☐	☐	☒	☒	☐	☐	☐	☐
mohan	☒	☒	☐	☐	☐	☒	☒	☐	☐	☐	☐
ricky	☒	☒	☐	☐	☒	☒	☐	☐	☐	☐	☐
sree	☐	☒	☐	☒	☐	☒	☒	☐	☐	☐	☐

Project-based matrix authorization strategy Level Security?

step1: Login as an administrator **just enable project level security.** (Overall read permission)

[illegible]

User/group	Overall	Credentials		Agent		Job		Run	View	SCM	Lockable Resources													
	Administrator	Read	Create	Delete	View	Build	Configure	Create	Delete	Discover	Move	Read	Workspace	Replay	Update	Configure	Create	Delete	Read	Tag	Reserve	Unlink	View	
 Anonymous Users	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	 
 Authenticated Users	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	 
Add user or group...																								

once you enabled you have to add the users

step2: Select the Job-> Configure.

☒ Enable project-based security

Inheritance Strategy: Inherit permissions from parent ACL

This item will inherit its parent items permissions (in addition to any permissions granted here). If this item is at the top level in Jenkins, it will inherit the [global security settings](#).

User/group	Credentials		Job		Run	SCM										
	Create	Delete	Update	View	Build	Cancel	Configure	Delete	Discover	Move	Read	Workspace	Delete	Replay	Update	Tag
Anonymous Users	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐
Authenticated Users	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐	☐

Add user or group...

step3: Click on the add user or group

grouping of jobs at view level.

Jenkins

- New Item
- People
- Build History
- Manage Jenkins
- My Views
- Lockable Resources
- New View

Build Queue

View name:

☐ Build Pipeline View
 Shows the jobs in a build pipeline view. The complete pipeline of jobs that a version propagates through are shown as a row in the view.

☒ List View
 Shows items in a simple list format. You can choose which jobs are to be displayed in which view.

☐ My View
 This view automatically displays all the jobs that the current user has an access to.

OK

Master Slave Architecture? Why we need? (Parallel running jobs)

UC: We have multiple jobs at same time.

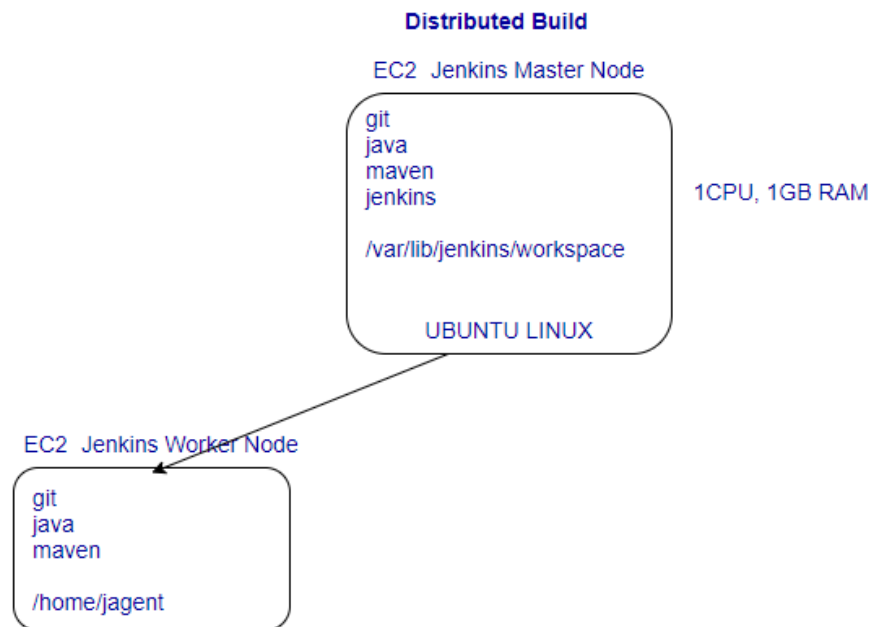
Recap: so far, we took a server (EC2 instance), install Jenkins on that server, running jobs on Jenkins's server, when we create a job it's created a workspace. Wherever you install the Jenkins is known as Jenkins's master (UI).

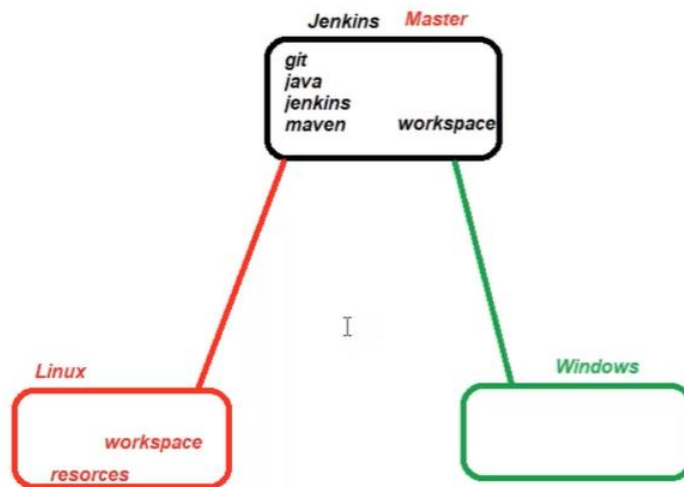
What is slave?

Slave always listen to the master, whenever master give instruction to the slave, slave executes those instructions. Once master is getting overloaded it needs some helping hands, so it extends its jobs and tasks to slave machines.e.g. If we install multiple applications on our mobile then mobile speed becomes slow, same happens with server also if load increases its performance getting slow down.

free -m

Once master is getting overloaded it delegates its workload to slave machines, if want to separate different types of tasks on different machine then also master delegates the task to slave based on the type of job.





Jenkins's master **pulls the code from remote GitHub repository every time there is code commit**. Jenkins master machine can delegate the jobs to agent server/machines, so that job will be run on agent server, which means that we are utilizing the resources of agent machine, on request from Jenkins's master, slave carry out builds and tests and produce test reports. Every resource has their own capacity can't go beyond their capacity.

Objective is to distribute the load in multiple servers, no need to install jenkins on agent servers.

Master delegates the jobs to slave, master just only manage the jobs.

How to add slaves to the Jenkins master?

By default, master does not accept any connection from external resources. There are different ways to establish a connection between master and slave.

Pre-requisite:

- Install java on all slaves.
- Install maven on all slaves (or use maven installed on Jenkins)

Dashboard > Nodes >

[Back to Dashboard](#)

- [Manage Jenkins](#)
- [New Node](#)**
- [Configure Clouds](#)
- [Node Monitoring](#)

Build Queue ^

No builds in the queue.

Build Executor Status ^

1	Idle
2	Idle

Name ?

Description ?

Number of executors ?

Remote root directory ?

Remote directory is mandatory

Labels ?

Usage ?

Use this node as much as possible

Launch method

- ✓ Launch agent by connecting it to the controller
- Launch agent via execution of command on the controller
- Launch agents via SSH
- Custom WorkDir path

Way1: Using Agents

- Create a home directory on all slaves.

L*

- cd home
- mkdir jagent
- chmod 777 jagent/
- cd jagent

*I

Agents

TCP port for inbound agents

☐ Fixed :

☒ Random

☐ Disable

Agent protocols

☒ Inbound TCP Agent Protocol/4 (TLS encryption)

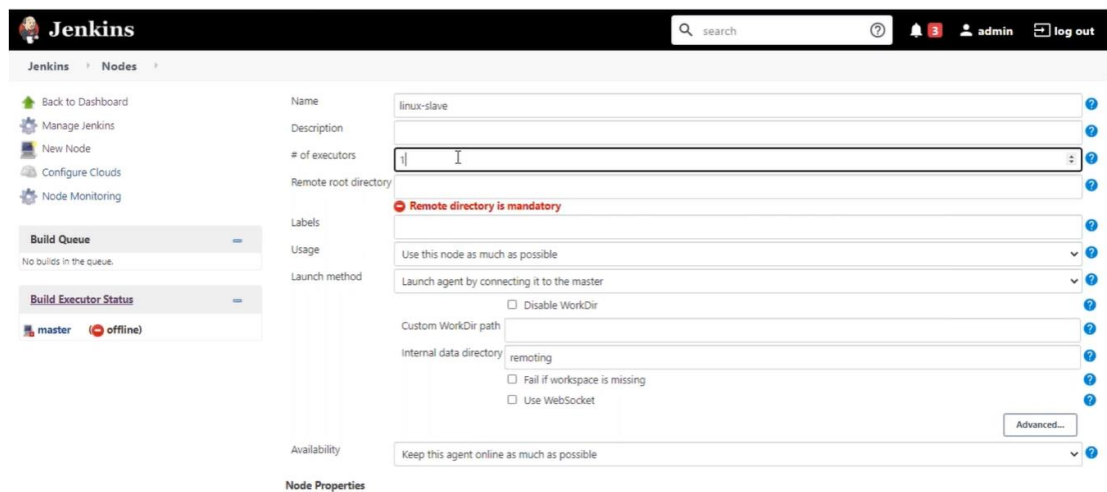
A TLS secured connection between the master and the agent performed by TLS upgrade of the socket.

CSRF Protection

step2: Add extra node (slave), tell Jenkins's master about slaves hey! Jenkins we are adding slaves for your help.

[illegible]

Click on new node



The image shows the Jenkins 'New Node' configuration page. The 'Name' field is set to 'linux-slave'. The '# of executors' is set to '1'. The 'Remote root directory' is empty, with a red error message 'Remote directory is mandatory'. The 'Launch method' is set to 'Launch agent by connecting it to the master'. The 'Internal data directory' is set to 'remoting'. The 'Availability' is set to 'Keep this agent online as much as possible'. The 'Node Properties' section is empty.

name: linux-slave

Of executors > The number of jobs that can be run in parallel on agent server

remote root directory: /home/jagent

where the workspace will be generated, home directory at slave just like a /var/lib/jenkins in master

labels: linux-slave

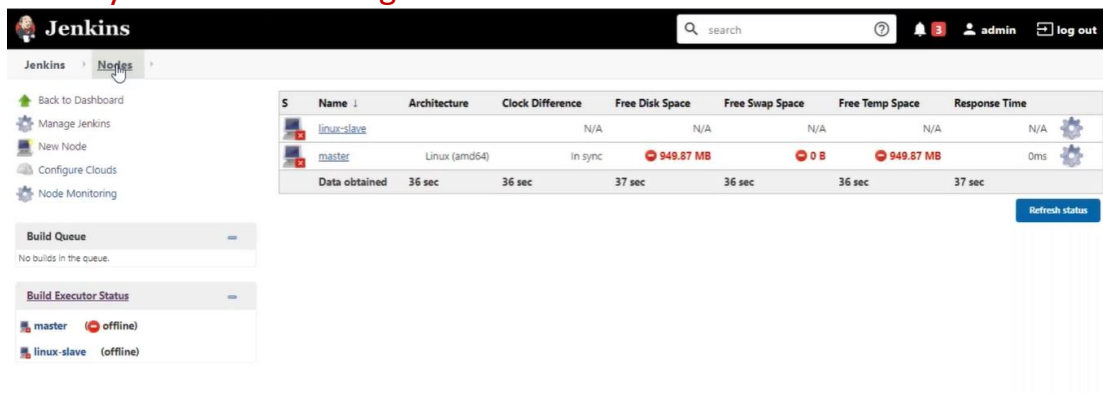
Usage: Use this node as much as possible (hey! master use this node first)

Launch method:

- **Launch agent by connecting it to the master.**
- Launch agent via execution of command on the master.
- Launch agents via ssh.

Availability: Keep this agent online as much as possible

When you save it linux-agent is still offline?



The image shows the Jenkins 'Nodes' page. The 'linux-slave' node is listed as 'offline'. The 'master' node is listed as 'online'.

S	Name	Architecture	Clock Difference	Free Disk Space	Free Swap Space	Free Temp Space	Response Time
	linux-slave		N/A	N/A	N/A	N/A	N/A
	master	Linux (amd64)	In sync	949.87 MB	0 B	949.87 MB	0ms
Data obtained		36 sec	36 sec	37 sec	36 sec	36 sec	37 sec

Click on linux-agent.

Jenkins > Nodes > linux-agent

Back to List

Status

Delete Agent

Configure

Build History

Load Statistics

Log

Build Executor Status

Agent linux-agent

Mark this node temporarily offline

Connect agent to Jenkins one of these ways:

- Launch agent from browser
- Run from agent command line

```
java -jar agent.jar -jnlpUrl http://10.128.0.18:8080/computer/linux-agent/slave-agent.jnlp -secret cabdda04fb20e3706b22bfff964892119997f900eaf8df59604a8e284ff29d8ea -workDir "/home/jagent"
```

Run from agent command line, with the secret stored in a file:

```
echo cabdda04fb20e3706b22bfff964892119997f900eaf8df59604a8e284ff29d8ea > secret-file
java -jar agent.jar -jnlpUrl http://10.128.0.18:8080/computer/linux-agent/slave-agent.jnlp -secret @secret-file -workDir "/home/jagent"
```

Projects tied to linux-agent

None

its looking for java and agent.jar in agent nodes.

- 1) Install the java
- 2) download the agent.jar
- 3) execute the command

right click on agent.jar and save (copy link address)

step3: In slave node

cd /home/jagent

wget <copied url>

```
root@ip-172-31-29-111:/home/jslave#
root@ip-172-31-29-111:/home/jslave#
root@ip-172-31-29-111:/home/jslave# wget http://18.219.69.64:8080/jnlpjars/agent.jar
--2020-07-12 17:39:16-- http://18.219.69.64:8080/jnlpjars/agent.jar
Connecting to 18.219.69.64:8080... connected.
HTTP request sent, awaiting response... 200 OK
Length: 1522833 (1.5M) [application/java-archive]
Saving to: 'agent.jar'

agent.jar          100%[=====] 1.45M  --.-KB/s   in 0.03s
2020-07-12 17:39:16 (52.3 MB/s) - 'agent.jar' saved [1522833/1522833]

root@ip-172-31-29-111:/home/jslave# ls -l
total 1488
-rw-r--r-- 1 root root 1522833 Jul 11 17:37 agent.jar
root@ip-172-31-29-111:/home/jslave#
```


click on new nodes-> Permanent Agent

name: linux-slave

of executors > The number of jobs that can be run in parallel on agent server

remote root directory: /home/jagent

where the workspace will be generated, home directory at slave just like a /var/lib/jenkins in master

labels: linux-slave grouping of slaves e.g., 3 servers are responsible for running frontend jobs and remaining 3 servers are responsible for running backend job(mysql)

Usage: Use this node as much as possible (hey! master use this node first)

Launch method:

- Launch agent by connecting it to the master.
- Launch agent via execution of command on the master.
- **Launch agents via ssh.**

Host->slave (will work because we have added the entry in /etc/hosts else enter ip)

Credentials-> username from **Add credential**

Host key verification strategy-> Non verification strategy

Save

Availability: Keep this agent online as much as possible

Add Credential

Home Page->Credentials->System->Global Credentials->add credentials

kind- ssh username with private key

Id - arbitrary name

Description->

username -> root // username login as

private key -> add private key of jenkins master

S	Name	Architecture	Clock Difference	Free Disk Space	Free Swap Space	Free Temp Space	Response Time
	master	Linux (amd64)	In sync	7.30 GB	0 B	7.30 GB	0ms
	Slave	Linux (amd64)	In sync	7.88 GB	0 B	7.88 GB	51ms
Data obtained			39 ms	19 ms	19 ms	18 ms	9 ms

Delegate the jobs to Slave

Job->Configure-> **General**->Restrict Where this project can be run

In Slave before running specific job

```
root@slave:/var/lib/jenkins# ls
remoting  remoting.jar
root@slave:/var/lib/jenkins#
```

In Slave after running specific job

Multiple Pipelines can be run from single master node, the output of the CI pipeline is, it gives you the artifacts.

Way3:

cd /home/devops

mkdir jslave

chown devops:devops /home/devops/jslave

The screenshot shows the Jenkins configuration page for a new slave agent. The 'Launch method' section is set to 'Launch agents via SSH'. The 'Host' field contains '35.188.20.254'. The 'Credentials' dropdown is set to 'devops/***** (forslave)' with an 'Add' button. The 'Host Key Verification Strategy' is set to 'Non verifying Verification Strategy'. An 'Advanced...' button is visible. The 'Availability' section is set to 'Keep this agent online as much as possible'. Under 'Tool Locations', the 'Tool Locations' checkbox is checked. The 'List of tool locations' section shows two entries: 1. Name: '(JDK) myjava', Home: '/usr/lib/jvm/java-8-openjdk-amd64', with a 'Delete' button. 2. Name: '(Maven) mymaven', Home: (empty field), with a 'Delete' button.

Launch method

Launch agents via SSH

Host

35.188.20.254

Credentials

devops/***** (forslave) Add

Host Key Verification Strategy

Non verifying Verification Strategy

Advanced...

Availability

Keep this agent online as much as possible

☐ Disable deferred wipeout on this node

☐ Environment variables

☒ Tool Locations

List of tool locations

Name

(JDK) myjava

Home

/usr/lib/jvm/java-8-openjdk-amd64

Delete

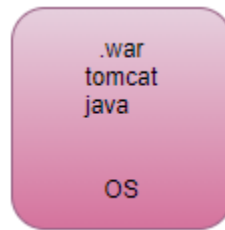
Name

(Maven) mymaven

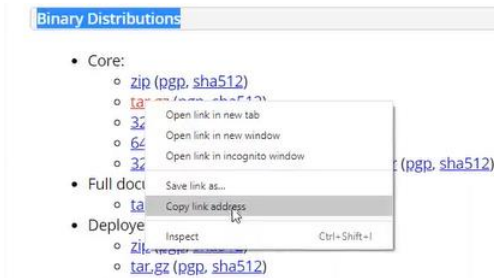
Home

Delete

Example: tomcat



- download tomcat server



- `wget <url>`
- `tar -xzf <name.gz> -C /opt // extract zip file`
- `cd /opt`
- `cd <name>/bin`
- `./startup.sh`
- `ps -es | grep tomcat`
- `<public Ip>:8080`
- `cd webapp`
- `cd /home/jagent/workspace/package/target/sampleapp.war .`
- `./stop.sh`
- `./startup.sh`
- `<public Ip>:8080/sampleapp/`